

Answers to tutor's follow-up questions

1. Why did soft voting improve recall but not overall accuracy versus clean stacking under nested CV?

Under repeated nested cross-validation, the proposed pipeline achieved a mean recall of 0.8944, compared with 0.8700 for clean stacking, an increase of about 2.44 percentage points. However, precision decreased from 0.8282 to 0.8100, while accuracy changed only from 0.8289 to 0.8267. Therefore, the gain in recall came at the expense of precision rather than an overall increase in correct classifications.

In practical terms, higher recall means the proposed model missed fewer positive cases, resulting in fewer false negatives relative to the number of actual positives. But the lower precision indicates it also produced more false positives among its positive predictions. Accuracy counts both false positives and false negatives as errors, so reducing false negatives does not necessarily increase accuracy if additional false positives offset that gain.

There is also an important distinction between AUC and accuracy here. The proposed method had a slightly higher mean AUC, 0.9068 versus 0.9014, which suggests slightly better ranking of positive versus negative cases across thresholds. But accuracy is calculated at the classifier's actual decision operating point, effectively the default binary decision threshold, so a better ranking does not guarantee higher accuracy at that particular threshold.

I would not claim that soft voting alone caused this change. My proposed method changed several components together: duplicate removal, mutual-information feature selection, hyperparameter tuning, the set of ensemble members, and the ensemble rule. Therefore, the experiment evaluates the proposed pipeline as a whole, not the isolated causal effect of soft voting.

2. How did mutual-information feature selection interact with the small (302-row) deduplicated set – any risk of selection leakage across folds?

Feature selection can cause leakage if performed once on the full dataset before cross-validation, since mutual information relies on the target labels. I specifically avoided that in my nested-CV implementation.

Mutual-information selection was placed inside each model's scikit-learn Pipeline. The candidate number of selected features, 8, 10, or all 13 predictors, was also treated as a hyperparameter. During the inner 4-fold GridSearchCV, the pipeline is fitted separately on each inner training fold. Therefore, the MI scores and selected features are calculated only from the inner training portion; the corresponding inner validation fold is not used to fit the selector. After the best configuration is chosen, the selected pipeline is refitted using the outer training data, and the outer test fold remains untouched until the final evaluation. This prevents feature-selection information from leaking from the outer test

fold into model selection. The report describes feature selection and hyperparameter selection as part of the inner modelling process.

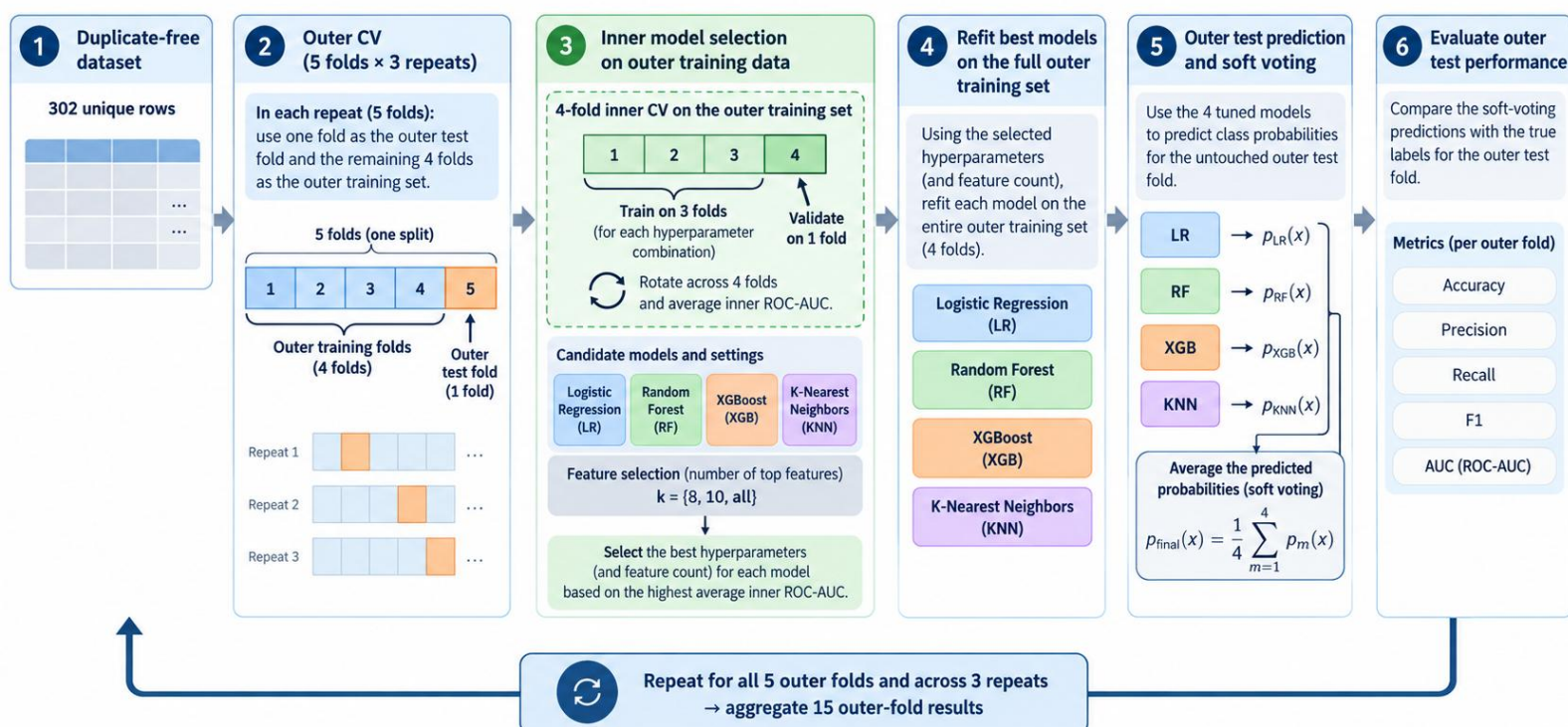
However, leakage and statistical instability are different issues. Although the procedure avoids selection leakage, the dataset is still very small after deduplication: only 302 unique rows, with 138 class-0 and 164 class-1 observations. With roughly 80% of the data in each outer training split, and only three-quarters of that available for an inner training split, MI estimates can be noisy. So selected features can change across folds.

In fact, that instability is visible in my results. LR sometimes selected all features and sometimes 10; RF usually retained all predictors; XGB usually selected 10; and KNN consistently selected a reduced subset. I interpret that as evidence that feature selection is model- and sample-dependent, rather than as evidence of a universally optimal subset.

So my conclusion is: I do not have MI selection leakage into the outer test folds, because selection is nested inside the training pipeline, but I do have increased feature-selection variance because the dataset is small. Nested CV helps ensure that this selection uncertainty is included in the reported generalisation estimate.

3. Explain your nested CV design (inner vs outer loops) and what the bootstrap intervals represent.

Repeated Nested Cross-Validation for the Proposed Soft Voting Pipeline 5-fold outer CV repeated 3 times = 15 outer test evaluations



The purpose of nested cross-validation is to separate model selection from performance estimation.

In my outer loop, I used stratified 5-fold cross-validation, repeated 3 times, yielding 15 outer train/test evaluations. For each outer split, approximately four-fifths of the 302 unique records form the outer-training set, and one-fifth form an untouched outer-test set. The outer-test fold is used only once, after all model and hyperparameter decisions for that split have been completed.

Within each outer training set, I used 4-fold stratified inner cross-validation. For LR, RF, XGB, and KNN, GridSearchCV evaluated hyperparameter combinations along with feature-selection choices of 8, 10, or all features. ROC-AUC was the inner selection metric. The best configuration for each base learner was then fitted to the outer training data and combined into the soft-voting ensemble.

Finally, the fitted ensemble was evaluated on the untouched outer test fold. Clean stacking was trained and evaluated on the same outer train/test split, allowing the two models to be compared on a paired, fold-by-fold basis.

Therefore, the inner loop answers, "Which model configuration should I choose?", while the outer loop answers, "How well does that entire model-selection procedure generalise to unseen data?"

After the 15 outer evaluations, I calculated a paired difference for each metric on each outer fold:

$\text{difference} = \text{Proposed Soft Voting} - \text{Clean Stacking}.$

That gives 15 paired differences for accuracy, another 15 for precision, and so on. I then resampled those 15 paired differences with replacement 10,000 times. For each bootstrap sample, I calculated the mean difference. The 2.5th and 97.5th percentiles of those 10,000 bootstrap means form the reported descriptive 95% interval.

For example, recall had a mean difference of +2.44 percentage points and a descriptive interval of approximately +0.21 to +4.85 points. Precision had a mean difference of -1.82 points, with an interval of approximately -3.34 to -0.41. In contrast, the intervals for accuracy, F1 and AUC crossed zero.

4. With 97.1% duplicate overlap on the original split, would patient-level grouping change your conclusions if IDs existed?

If reliable patient IDs existed, I would prefer patient-level group splitting over simple row-level splitting, for example, using a group-aware cross-validation strategy in which all records from the same patient are kept entirely in either the training or test fold.

The reason is that my audit found that 199 of 205 test records, or 97.1%, had an identical predictor vector in the training set under the original row-level split. That demonstrates extremely high record-pattern overlap, but without patient IDs, I cannot determine whether those rows are repeated copies, repeated visits from the same patients, or whether some identical feature vectors genuinely belong to different patients.

Patient grouping would answer: can the model generalise to a completely unseen patient? It would ensure that information from a single patient does not appear in both the training and testing sets, even when two visits from that patient are not identical.

I would expect patient-level grouping to make the evaluation more reliable if many of the duplicate or near-duplicate records actually came from the same patients. In that case, the original random row split could give an overly optimistic estimate because records from the same patient might appear in both training and testing sets. However, I cannot say that performance would definitely decrease under patient-level grouping, because the dataset does not include patient IDs or sufficient information to confirm the source of the duplicate records.